

Timestamp Ordering Protocol (TO Protocol)

Comprehensive, High-Yield Concurrency Control Study Notes

1. What is Timestamp Ordering Protocol?

Timestamp Ordering Protocol (TO Protocol) is a pessimistic concurrency-control mechanism used by DBMS to determine the serializability order of executing transactions. Each transaction is assigned a unique, monotonically increasing identifier called a **timestamp**.

★ Core Principle

Older transactions get strict priority over younger transactions.

If $TS(T_1) < TS(T_2)$, the protocol guarantees that the execution schedule will be equivalent to the serial schedule: $T_1 \rightarrow T_2$.

Transaction Ordering Example:

$T_1 \rightarrow TS(T_1) = 100$ (Oldest)

$T_2 \rightarrow TS(T_2) = 200$ (Younger)

$T_3 \rightarrow TS(T_3) = 300$ (Youngest)

Target Serial Execution Order: $T_1 \rightarrow T_2 \rightarrow T_3$

2. Timestamp Characteristics

The timestamp value $TS(T_i)$ is assigned when transaction T_i enters the system.

- **Monotonicity:** Newer transactions always receive higher timestamps.
- **Implementation:** Timestamps do not strictly require system clock time; logical counters/sequence numbers are standard.
- **Comparison:** If $TS(T_1) = 100$ and $TS(T_2) = 200$, then T_1 is older and T_2 is younger.

3. Three Critical Timestamps to Track

To evaluate schedules under TO Protocol, track three timestamps per data item A :

① $TS(T_i)$

Transaction Timestamp:

Assigned uniquely to transaction T_i at start.

② $RTS(A)$

Read Timestamp: Largest

timestamp among all transactions that successfully performed $READ(A)$.

③ $WTS(A)$

Write Timestamp: Largest

timestamp among all transactions that successfully performed $WRITE(A)$.

4. Three Important Conflict Cases

Conflicts occur when an older transaction attempts to perform an operation after a younger transaction has already executed a conflicting operation on the same data item.

Scenario & Sequence	Protocol Condition	Action / Result	Reasoning
Case 1: Older READ → Younger WRITE T1(100): R(A) T2(200): W(A)	$TS(T_1) \geq WTS(A)$	ALLOW ✓	Order matches serialization priority ($T_1 \rightarrow T_2$). Update $WTS(A) = 200$.
Case 2: Younger READ → Older WRITE T2(200): R(A) T1(100): W(A)	$TS(T_1) < RTS(A)$	ROLLBACK T ₁ ✗	Older transaction attempts write after a younger transaction already read the older version.
Case 3: Younger WRITE → Older WRITE T2(200): W(A) T1(100): W(A)	$TS(T_1) < WTS(A)$	ROLLBACK T ₁ ✗	Older write overwrites a newer value, causing serialization inconsistency.

💡 Memory Trick for Conflicts

Younger READ or WRITE happened FIRST → Older conflicting operation comes LATER → ROLLBACK requesting transaction!

Note: **Read-Read (R-R)** is NEVER a conflict. Multiple transactions can freely read data item *A* without causing rollbacks.

5. Protocol Comparison: Timestamp Ordering vs. 2PL

Feature	Two-Phase Locking (2PL)	Timestamp Ordering (TO)
Primary Mechanism	Locks (Shared / Exclusive)	Transaction Timestamps
Phases	Growing & Shrinking Phase	None (Operation-by-Operation)
Deadlock Risk	Possible (Requires detection/prevention)	Deadlock-Free (No lock waiting)
Starvation/Rollback	Cascading rollbacks possible (if unstrict)	High rollback frequency under heavy conflict
Serialization Order	Determined by Lock Point	Determined strictly by assigned TS